

THE APPLIER

Bitcoin Machine Code, days 41 to 57: the state of the project on
2026-10-07

A follow-up to "21 FOR 21"

Bitcoin Machine Code · github.com/BobClawblaw/bitcoinmachinecode

The fourth report in the series. "21 FOR 21" covered 2026-08-11 to 2026-09-02. "The Measuring Equipment" covered 2026-09-02 to 2026-09-19, and "Run 28 vs Core" the clean benchmark pair that ended on 2026-09-21. This one runs from 2026-09-21 to 2026-10-07. It was compiled from the project's own records: the git history of <https://github.com/BobClawblaw/bitcoinmachinecode>, the pull requests, the worklogs and the performance plan, docs/releases, docs/reports (the IBD and stage reports), docs/devlog (the deployment record and the incident notes), and the assistant's persistent memory notes. Every quote is verbatim from one of those, and every figure names where it came from. As before, the counts are true as of the date given and will drift.

```
"The win is concurrency, not a faster per-block apply."  
-- docs/reports/2026-10-05-logged-ibd-pair.md
```

Contents

PROLOGUE: WHERE THE LAST REPORTS STOPPED.....	4
PART I: THE NUMBERS – what seventeen days added.....	5
Chapter 1: Counting	5
Chapter 2: Who wrote it	5
PART II: THE LINK – the benchmark that measured the wrong thing well.....	6
Chapter 3: Run 29, and what a repeat is for (09-22)	6
Chapter 4: The repair (09-28 to 09-29)	6
Chapter 5: A one-second mount, nine hours lost (09-29)	6
Chapter 6: The even comparison, and what it cost (09-29 to 10-01)	7
Chapter 7: Core's own rules (10-04)	7
PART III: THE OUTAGES – what a deploy proves.....	8
Chapter 8: 70 minutes on one block (09-25)	8
Chapter 9: 23 hours, and an address that named its bug (10-02 to 10-03)	8
Chapter 10: The service was up and RPC was not (10-07)	9
PART IV: THE LOGGED PAIR – concurrency, not apply speed.....	10
Chapter 11: Run 34 against Core rerun #6 (10-04 to 10-05)	10
Chapter 12: The 78 minutes that were one deaf worker (10-05)	10
PART V: THE PLAN – making the applier faster than Core's.....	12
Chapter 13: The ask (10-05)	12
Chapter 14: Taking work off the applier (10-05 to 10-06)	12
Chapter 15: The same code, three different put times (B11)	13
Chapter 16: A two-second sleep (B9 part 2, 10-07)	13
Chapter 17: Run 41	14
Chapter 18: The category Core keeps	14
PART VI: PARITY – what Core heard, and what it ignored.....	16
Chapter 19: No transaction we relayed ever reached Core (10-02)	16
Chapter 20: The compact blocks we served (09-25)	16
Chapter 21: Core's words, byte for byte (10-01 to 10-03)	16
Chapter 22: Crypto parity, and two bench rows that flattered us (09-27 to 09-28)	17
Chapter 23: The first sync that checked every script (09-27)	17
PART VII: THE SCAR TISSUE – patterns, and the rules they left.....	18
Chapter 24: Per-something costs that were not per-something	18
Chapter 25: The box as an adversary	18
Chapter 26: The port that said "applies to x86"	18
Chapter 27: The rules now in force	19
PART VIII: WHAT IS STILL OPEN.....	20
APPENDIX A: THE NUMBERS.....	21

PROLOGUE: WHERE THE LAST REPORTS STOPPED

"21 FOR 21" ended on a node, written entirely in hand-crafted x86-64 assembly and C with no human-typed code, whose UTX0 set was MuHash-identical to Bitcoin Core's. "The Measuring Equipment" spent seventeen days making the benchmarks believable. "Run 28 vs Core" gave the first clean number: a from-genesis mainnet sync in 18 h 24 m 02 s against an unpolled Core v31.1's 19 h 32 m 54 s, 5.9% in bmc's favour, the UTX0 set identical at 968,025.

That result had a qualifier which turned out to matter more than the number. Both nodes were bound by the same ~88 Mbit/s link. A 5.9% margin on a link-bound race says the node is not slower than Core. It does not say how fast it is.

The short version of the state on 2026-10-07:

- Speed. On a repaired link and on Core's own download rules where it counts, bmc run 41 had every index at the tip in 3 h 48 m 22 s. Core v31.1 rerun #7, on the same machine, the same NVMe and the same indexes, took 9 h 50 m 04 s. That is 0.39 of Core's time, and bmc was faster in every one of the ten 100,000-block segments (docs/reports/2026-10-07-run41-vs-core7-stage-report.md).
- What the margin is made of. On 10-05 a fully logged pair showed that bmc's per-block apply was 23% slower than Core's, and that the whole lead came from downloading and applying at the same time. The next three days went into the applier. In run 41 it is 11% faster than Core's: 11,842 thread-seconds against 13,340.
- Correctness. Every full-sync capstone in the period matched Core: runs 28 to 41, and the first sync with `assumevalid=0`, in which every script was evaluated. The UTX0 set of run 41 is MuHash-identical to Core's at 970,364.
- Memory. This is the category Core keeps. Run 41's PSS peaked at 75.6 GB against Core's 34.3 GB, with the heap at 32.2 GB against 12.0.
- Production. It was down twice: 23 hours on 10-02/03, and RPC alone for 2 h 17 m on 10-07. Each outage added a line to what "a verified deploy" means.
- The work. 107 merged pull requests and 193 non-merge commits since 2026-09-21, written by Claude Opus 5.5 and Claude Fable 5.1, with one human operator deciding, merging and deploying.

The title is literal. "The applier" is the thread that connects each block to the UTX0 set. For most of these seventeen days it was the thing nobody had measured properly. When it was measured, it was the thing that was slower than Core. By the end it was faster.

PART I: THE NUMBERS – what seventeen days added

Chapter 1: Counting

Counted on `main` from the last commit of 2026-09-21 (`7c6a67fc`, 17:58Z) to `aa22622b` (PR #404), the method the previous reports used:

merged pull requests	107 (#291 to #404, with gaps; #405 open)
non-merge commits	193
files changed	358, +35,769 / -2,322 lines
test programs (<code>asm/tests/test_*.c</code>)	427 → 456
full mainnet syncs completed, bmc	runs 29 to 34 and 37 to 41, plus the <code>assumevalid=0</code> sync
full mainnet syncs completed, Core v31.1	reruns #4 to #7; #2 and #3 cut short
production snapshots deployed	46 (<code>deploy-20260925b</code> to <code>deploy-20261007a</code>)

The deployment count is from `docs/devlog/DEPLOYMENT_HISTORY.md`, backfilled on 10-07 from the commit each snapshot binary stamps into itself. Five production restarts in that window left no snapshot on disk at all, one snapshot was built from a dirty tree, and two were written and never run. The record says which.

Chapter 2: Who wrote it

From the `Co-Authored-By` trailers of the 193 commits: Claude Opus 5.5 114 (45 of them in its 1M-context form), Claude Fable 5.1 67, Claude Opus 5 2, and 10 with no trailer. Opus 5.5 arrived on 09-24 and Opus 5 left the same week.

The operator's role did not change from the previous report: every merge to `main` and every production deploy is the operator's decision. What changed is how often it was made. On 09-28 production took eight deploys in one day (`a` to `h`). On 10-06 four benchmark syncs ran back to back, and a dozen pull requests landed between them.

A second tree joined the work: `bmc_osx`, a macOS/AArch64 port kept as a branch of the same repository. It found four real defects in the shared code (Part VI) and was wrong four times about whether its fixes applied to x86 (Part VII).

PART II: THE LINK – the benchmark that measured the wrong thing well

Chapter 3: Run 29, and what a repeat is for (09-22)

Run 29 repeated run 28 on the same commit against the same Core baseline: 18:28:54, 5.5% ahead, PASS at 968,154 (docs/reports/2026-09-22-run29-vs-core.md). Its purpose was the spread. Two bmc runs of the same code differed by 4 m 52 s, so a margin of an hour was real and a margin of five minutes was not. Both runs were bound by the link: ~88 Mbit/s of peer traffic sustained, for both nodes, the whole way. On that link, how fast either node applied blocks hardly mattered.

Chapter 4: The repair (09-28 to 09-29)

On 09-28 the operator repaired the box's backhaul. On 09-29 from about 13:30Z it moved to a faster one, measured at 636 Mbit/s sustained inbound (docs/reports/2026-09-30-ibd-benchmarks-bmc-vs-core.md). Nothing in the node changed, and every number moved:

	old link (09-22)	repaired link
bmc to 400,000	92 min (run 29)	24 min (run 30)
bmc to 968,987	–	4:54:30 (run 30)
Core to 968,987	–	11:01:00 (rerun #4)

Run 30 was 2.24× faster than Core. On the old link it had been 1.06×. The eighteen-hour syncs of the previous report were the link's numbers. The implementations had been waiting on it equally.

Chapter 5: A one-second mount, nine hours lost (09-29)

Core rerun #2 started at 16:02Z on 09-28. At 01:40Z on 09-29 a test in a different project on the same host (the ClusterMan VM tests) ran `mount -o ro ... /mnt; umount /mnt`, and the mount was up for about a second. The benchmark NVMe was mounted at `/mnt/nvme8tb`, under `/mnt`. For that second, Core's block write saw a read-only filesystem. Core stopped with EROFS at 924,616 after 9 h 38 m 24 s.

Two rules came out of it: read the journal for mount and loop events before blaming a drive, and run timed datadirs from `/srv/nvme8tb`, a bind mount that no `/mnt` mount can shadow (memory note `mount-on-mnt-shadows-nvme8tb.md`). Rerun #3 was cut by the operator when the backhaul changed underneath it. Rerun #4 started at 13:54Z on the new link and became the baseline for the next week.

Chapter 6: The even comparison, and what it cost (09-29 to 10-01)

With the link out of the way, the differences in configuration were all that was left, and on 09-29 every setting was checked line by line (docs/devlog/BENCHMARKS.md, "The even comparison"). The list found three that favoured Core:

- Download peers. Runs 27 to 30 used 8, because "Core's is 8". Core's `MAX_OUTBOUND_FULL_RELAY_CONNECTIONS` is 8, but during IBD Core downloads from every outbound peer that can serve blocks, which includes its two block-relay-only peers: ten (docs/CORE_DIVERGENCES.md). The register had read one constant and missed the other.
- The download window: Core requests no further than 1,024 blocks above its connected tip, while bmc's window was 4,096.
- `dbcache`: the default was 1024 MiB here, "a value no Core release defaults to". Core's is 450.

All three were matched (#344, #347). Run 31 on the matched settings came in at 5:28:11, 2.01× Core #4. Against run 30 that was a loss, and the report named it: "the 4,096 window was worth 11.5% to bmc". A Core-sized window starved bmc's 40-block chunk requests. The fix was to stay at 1,024 and ask for less at once: 16-block chunks, A/B-tested to 300,000 blocks at 19.5% and 14.1% faster than 40 (#361, 10-01).

The same days produced the parity correction behind every later number. On 10-01 the project went to one Core oracle, the v31.1 release, on RPC 8335 (#364). The v31.99 development build that had answered on those paths was deleted. Its answers had invented work, because they differed from the version bmc targets.

Chapter 7: Core's own rules (10-04)

Matching Core's settings still left bmc's own download policy in the race: peers ranked by speed, slow peers rotated, a staller banned. That is a real feature, but it does not compare implementations. So on 10-04 bmc learned to download by Core's rules (`bmc.dlshape=core`, #380): 16 blocks in flight per peer, topped up as each lands, peers in random order, no ranking or rate floor, a staller disconnected and never banned, and Core's block download timeout. A per-block stage log came with it (`bmc.benchlog`), which splits each block's apply into the same columns on both nodes. A `[ready]` line marks the moment every index reaches the tip.

The comparison was now as even as the project could make it. Part IV is what it showed.

PART III: THE OUTAGES – what a deploy proves

Production ran three incidents in this period. Each passed every check that existed at the time, and each added one.

Chapter 8: 70 minutes on one block (09-25)

PR #303 made bmc's legs ask for headers from the parent of the announced block, as Core does. It was deployed, checked for what it changed, and called good. On the next block, production's apply stalled for 70 minutes at 968,555.

The cause was two `close()` calls on the same descriptor in the boot fill. The first closed it, the kernel reused the number for the store's cached block file, and the second closed the store's file. The Mac session's note found it ([worklog/2026-09-25-note-for-x86.md](#)), and #306 fixed it.

The rule: a deploy is verified when a new block arrives and is applied with the oracle's hash, not when the change works (memory note [deploy-watch-the-next-block.md](#)).

Chapter 9: 23 hours, and an address that named its bug (10-02 to 10-03)

`deploy-20261002a` applied block 969,530 correctly at about 04:00Z and was called verified under the new rule. At 04:02Z it segfaulted. It crashed again about 40 seconds after every restart, as soon as mempool.space re-synced through the Esplora facade. At 04:28Z systemd gave up, and nothing noticed until the next session, 23 hours later.

The cause was a call path that skipped a lock. #368's facade batch route called `rpc_chain_tx_blockhash` without the execution lock while `irs_refresh`, on another thread, was zeroing the run maps it reads.

The address found it first. The unit runs with `LimitCORE=0`, because the decrypted wallet seed is in memory and a core file would put it on disk. There was no dump to open:

```
"All eleven crashes faulted at the same address, 0x6a61e3bd8. That address is exactly the run's sparse-index offset (48 + 1,425,612,630 × 20) plus the first binary-search probe (16 × 2,784,399) on top of a NULL base."
```

```
-- docs/releases/2026-10-03-the-outage-and-cores-words.md
```

A constant address despite ASLR, equal to a data-derived offset, means a NULL pointer plus an index: a map read after it was cleared. #373 fixed the lock. #376 added a crash handler that prints the fault address, the registers and a backtrace to the log, which `addr2line` resolves against the non-PIE binary, still without a core file. #379 fixed six more instances of the same three defect classes, found by auditing for them.

Two rules: one good block proves the apply path and nothing else, so a deploy also needs `NRestarts=0` ten minutes in with the RPC clients reconnected; and anything that calls into the chain code directly must take the lock the RPC dispatcher would have taken (memory note `lock-free-callers-bypass-the-exec-lock.md`).

Chapter 10: The service was up and RPC was not (10-07)

At 16:26:22Z on 10-07 production logged one line:

```
[rpc] accept: out of file descriptors (Too many open files) -- backing off; the RPC listener is degraded
```

and nothing watched for it. P2P sync carried on, the tip advanced, `systemctl is-active` said active. The RPC port and the Esplora facade accepted nothing, and the mempool.space instance built on them stopped at 970,365. The operator asked "why is production down?" at 18:22Z.

`/proc` had the answer: 1,023 of 1,024 descriptors open, 314 of them on `index.dat`, each paired with two block files. The getblock reader lane, part of the 10-05 work to stop RPC lock-ups (Part V), gave each thread a private store handle (three descriptors) and an 8 MB buffer, in thread-local storage, the first time it called `getblock`. For the fixed pool of RPC workers that is a fixed cost. The Esplora facade is not a pool. It starts a thread per connection, and mempool.space reads blocks through it all day. Every one of those threads exited holding its lane.

The fix is a pthread key whose destructor releases the lane when its thread exits (#402). Its test runs 64 threads that each call `getblock` once and exit, and checks that the descriptor count does not move. On the unfixed code it went from 14 to 206. Production was back at 18:43:46Z on `deploy-20261007a`, after 2 h 17 m. The descriptor count has stayed at 66 since. A fourth criterion joined the deploy check: the descriptor count stays flat across client traffic. That is a measurement over time, and the only one of the four that would have caught this (`docs/devlog/INCIDENT_2026-10-07_reader_lane_fd_leak.md`).

The 09-03 codebase audit had described this failure shape for a different leak, a month earlier: at the default limit of 1,024, "the RPC listener and the inbound P2P listener both stop accepting".

PART IV: THE LOGGED PAIR – concurrency, not apply speed

Chapter 11: Run 34 against Core rerun #6 (10-04 to 10-05)

Core rerun #6 started at 18:20:24Z on 10-04 with the stage logging on. Run 34 followed on Core's download rules. Both reached the point where every index was at the tip:

	Core v31.1 rerun #6	bmc run 34
ready	10:42:05	7:17:39
applier busy (s)	13,909 (36%)	17,166 (64%)
applier waiting (s)	24,587 (64%)	9,757 (36%)

1.47× faster on equal download terms, and the reason was not where anyone had assumed. The report's second finding:

```
"The win is concurrency, not a faster per-block apply. Core serialises download
and connection on one thread (13.9k busy + 24.6k waiting); bmc overlaps them
(17.2k busy, 9.8k waiting, most of that in the last segment)."
```

-- docs/reports/2026-10-05-logged-ibd-pair.md

Core runs `ProcessNewBlock` on its message-handling thread, so downloading and applying take turns. bmc downloads in forked workers while the applier connects blocks, so both happen at once. That overlap was worth more than bmc's whole per-block deficit. And there was a deficit: the applier was 23% slower per block than Core's, mostly in writing to the UTXO store (`put`, 7,812 s of the 17,166).

The memory note written that day says how to talk about it:

```
"claim apply speed only with the stage row beside it"
```

-- memory note ibd-win-is-concurrency-not-apply.md

Chapter 12: The 78 minutes that were one deaf worker (10-05)

The pair report blamed run 34's slow last segment on random peers: "download-bound under random peers". Run 35, launched to check the next change, showed a different shape. One worker held the oldest missing chunk. The stall rule evicted it with `SIGUSR1` twelve times, and it never acted on the signal. The window sat for seven minutes with nine workers idle at its edge and the applier at zero lag. Run 34's log had the same shape four times, about twenty minutes each, 78 minutes in all.

#384 made the first eviction hand the chunk to an idle worker straight away, as Core re-requests a disconnected staller's blocks. The fix's own diagnostic

lines then misreported twice: one said "did not answer" with the worker's acknowledgement printed a line above, and another printed a stale "acknowledged late". #385 fixed the accounting. The memory note:

```
"check the [dlc wN] ack beside every stall line and block gaps > 60 s before  
calling a run download-bound"  
-- memory note eviction-is-a-signal-check-cursorhelp.md
```

PART V: THE PLAN – making the applier faster than Core's

Chapter 13: The ask (10-05)

"resolve the rest of the performance holes and any rpc improvements so we don't lock up so much compared to core."

-- the operator, *worklog/2026-10-05-performance-holes-plan.md*

The plan that answered it set numbers before work. Every item had to come with a measurement that moves, a test that fails with the change reverted, a full gate and a `--no-ff` merge. The targets, and where they ended:

target	start (run 34)	target	run 41
the applier (s)	17,166	≤ 12,000	11,842
UTX0 put (s)	7,812	≤ 4,500	3,842
headers to block 1	306 s	≤ 90 s	48 s
ready, ranked peers	5:28	≤ 4:45	3:48:22
exec-lock waits ≥ 2 s, production	~10 a day	0	0

Chapter 14: Taking work off the applier (10-05 to 10-06)

In order of landing (each in docs/releases and the plan's status register):

- B1+B2 (#383): the undo record for a spent input reuses the prevout the validation phase had already resolved, instead of looking it up again. 2.30 → 1.13 μs per spent input on the bench; put 7,812 → 5,687 s in run 37.
- B5 (#383): the first header page is asked of four peers at once, and the fastest leads. Headers to block 1: 5:06 → 1:58.
- The RPC lanes (#383): the Esplora facade had taken the exclusive execution lock for every dispatch. Now it takes what the method's class needs. A txindex lane and a getblock reader lane serve the hot calls without the lock. Production's waits of 2 s or more went from about ten a day to none (and the reader lane is Chapter 10's leak).
- B4 (#390): txindex, the block filter and the address and spender tails are built by a forked index worker fed from a 1,024-slot ring: 688 s off the applier. Run 38: 4:17:09, 33 minutes faster than run 37.
- A7/A8 (#387, #388): `getrawtransaction` at 32 concurrent clients went from 382 ms to 5 ms, Core's figure, and `getdeploymentinfo`, which re-walked about 10,000 headers per call under the exclusive lock, from 2 s to 4 ms.
- B8 (#395): the cumulative-work records are appended as the download goes instead of in one pass at the end. Run 38 had paused for about a minute

there, on 970,000 cold 80-byte reads at queue depth 1. After B8 the pause was 0.55 s.

- B3 (#396): the UTXO memtable flush runs in a forked writer while the applier carries on. Flush time on the applier: 1,357 → 131 s. Building it found a latent defect in recovery: the reload's tombstone pass started at byte 0 instead of the checkpoint's offset.
- M1 (#394): the sampler had reported 26 GB of anonymous memory. It was summing copy-on-write pages once per forked child, 13 times. The real figure was about 10 GB.

Chapter 15: The same code, three different put times (B11)

Three runs of the same put code reported three different put times: 4,994 s (run 38), 5,661 s (run 39), 7,135 s (run 40). Nothing in the put path had changed between them. The writer child of B3 was alive in run 40, so it was the obvious suspect. The per-input insert cost was the same with the writer alive and without it.

The tree's CPU time had risen only by the writer's own seconds. The extra 1,475 s had been spent waiting, not computing (plan, B11). The memtable's table and blob were `MAP_SHARED` mappings of files: every insert dirtied a file page the kernel would write back, and on a box with 8 GB of swap full and 85 GB of page cache, a reclaimed page came back from the file on the next touch. A file-backed mapping also gets no transparent huge pages, so a 1.6 GB random-access table paid a TLB miss per probe.

Nothing read those files back. The boot and every tool rebuild their view from the WAL. So B11 (#400) moved the table and blob into anonymous memory with `MADV_HUGEPAGE`. The files are still created at full size, because the offline tools read their sizes, and the table's first page stays mapped from the file, because the transaction-validation snapshot checks the live count there. In run 41 put was 3,842 s against run 40's 7,135. The insert went from 2,621 s to 664 and the spent-output delete from 3,079 s to 1,822, "by changing nothing in the code that runs them" (the release report, §2).

Adding that column exposed an older error. The release report's "undo capture" figures for runs 38 to 40 (2,242 → 2,463 → 3,079 s) had been printed one column off. They were the delete column. Undo capture had been 921 to 979 s throughout and had never been the mover. The report now says so at every place the shifted label survived.

Chapter 16: A two-second sleep (B9 part 2, 10-07)

Run 40 was ahead of Core #7 by the end, and behind it at the start: 2.0× slower to 100,000 and 1.7× slower to 200,000. The first hypothesis was round trips on tiny blocks. A rolling per-block request mode was built to test it (`bmc.dlccrollbelow`), and it A/B-tested 13% slower or level, so it was rejected.

Run 40's own log had a pattern in it. Below 100,000, chunk completions came in 99 bursts, 2.09 s apart, with nothing at all arriving in 76 of 210 seconds. The workers filled a 1,024-block window of tiny early blocks in about 0.3 s, then waited out the download loop's whole 2-second idle sleep before the window moved. The fix polls the committer's tip every 20 ms while it waits (`c6884a22`). The gate line read `idle waits cut short 1670 (B9)`. 100,000 came at 1:35 instead of 4:16.

B10 landed in the same PR. Right after run 40's download, two legs claimed a height of 975,945 on a 970,229 chain. That started a full re-probe and re-ranking of 161 peers for 37 real blocks. A claim more than 50 blocks plus a block a minute above the pool's last median is no longer believed, and the tail went from 99 s to 34.

Chapter 17: Run 41

Run 41 launched at 12:52Z on 10-07 on `7027c734`, the commit with B9 part 2, B10, B11 and the B3 reap. From the stage report:

height	Core v31.1 rerun #7	bmc run 41	bmc / Core
100,000	2:06	1:35	0.76
200,000	4:44	2:55	0.62
300,000	15:30	6:57	0.45
500,000	1:43:30	44:25	0.43
700,000	4:18:21	1:42:57	0.40
900,000	8:13:37	3:10:02	0.38
every index at the tip	9:50:04	3:48:22	0.39
CPU, all processes	13 h 15 m	7 h 39 m	0.58
the applier (thread-seconds)	13,340	11,842	0.89
UTXO set	MuHash at 970,333	identical at 970,364	

Core #7 was itself 52 minutes faster than #6, all of it before 500,000. Its connect columns were within 4% of #6's, so "the difference is the peers it was given, not the box" (the release report). bmc was compared against the faster of the two.

The full progression of the release line against Core: run 34 0.68 on Core's download rules; runs 37 to 40 0.40 to 0.48 on bmc's; run 41 0.39.

Chapter 18: The category Core keeps

	Core #7	run 41
heap, mean / peak	10.0 / 12.0 GB	15.3 / 32.2 GB
PSS, mean / peak	17.6 / 34.3 GB	39.9 / 75.6 GB

Some of that is B11: the memtable's pages moved from the file-backed column to the anonymous one, counted once instead of hidden in page cache. The rest is the design. bmc keeps forked workers, a staged chunk store and a full memtable alongside each other, where Core keeps one process and a bounded cache. The release note states it plainly as the one category Core keeps, and `bmc.memtableanon=0` restores the old mapping for anyone who wants the old trade.

PART VI: PARITY – what Core heard, and what it ignored

Chapter 19: No transaction we relayed ever reached Core (10-02)

bmc's handshake sent BIP339 `wtxidrelay`, and its relay code announced transactions by `txid` (`MSG_TX`). A comment said `txid` announcements were "universally understood". They are not:

```
"once wtxidrelay is negotiated, Core's net_processing ignores MSG_TX invs from
that peer ("Ignore INVs that don't match wtxidrelay setting"). tx_relay.c and
txann.c announce MSG_TX, so no transaction bmc relayed or originated ever reached
a Core peer -- production included."
-- commit 5c18b05c
```

It was seen on a regtest run where Core's `getpeerinfo` showed 183 bytes of `inv` from bmc and no `getdata` back. The tests had asserted that `wtxidrelay` was sent: they pinned the defect. The bumpfee end-to-end run went from 47 ok and 18 failed to 64 and 1. The next day bmc announced by `MSG_WTX` properly and kept BIP339 (#374).

Chapter 20: The compact blocks we served (09-25)

```
"cmpctblock_build stored only the low byte of ntx-1 (`mov byte [r12+88], al`) with
the short ids at +89, so for any block of 254 or more transactions -- nearly all
of mainnet -- the cmpctblock served to an inbound peer (bitcoin_serve.asm:1084)
was malformed"
-- commit 64c5102b
```

The Mac port found it while porting the encoder, and the test it brought uses block 481,827 (1,377 transactions) as a fixture. On the receiving side a similar defect had been fixed on 09-09: a 5-byte compare of "block" had swallowed every `blocktxn` for three days. Compact blocks are a protocol where the sender is rarely checked, because the receiver falls back to the full block and nothing visibly breaks.

Chapter 21: Core's words, byte for byte (10-01 to 10-03)

BlockYard, the project's block explorer, ran a mempool differential against Core and found bmc's pool missing children Core accepted (15 and 14 of 35, two of 246 clusters). That moved the mempool to Core v31.1's policy wholesale (#363): cluster limits only, v31.1's replacement rule with the feerate-diagram check, optimal linearization, `lp1c` package RBF, fee floors that round up as v31.1's do. Rejections followed (#375, #377, #378). For 26 rejection cases, bmc now returns Core's exact reject text in Core's stage order, byte for byte.

Chapter 22: Crypto parity, and two bench rows that flattered us (09-27 to 09-28)

Every file in Core's `src/bench` was classified and paired where bmc had a counterpart. The `secp256k1` work (#319, #320) brought ECDSA verify to 20.7 μ s and BIP340 to 22.4 μ s, against `libsecp256k1`'s 21.0 and 22.0. Asm ports of the modules (#329 to #334) took MuHash finalize from 1,861 μ s, 66 $\times$ behind Core, to 27.9 μ s against Core's 28.0.

Two rows were published too well. MuHash's inverse was posted as "1.18 $\times$ ahead", and a multiply as "7.7 $\times$ ". Decomposed, Core's benchmark loop does `Finalize` plus a `/=`, two multiplies that bmc's row skipped, so the inverse was level. The multiply row compared one of bmc's multiplies against Core's `*=`, which runs two: the honest figure was 3.8 $\times$. The memory note:

```
"decompose both sides before claiming a win"  
-- memory note benchmark-rows-copy-cores-loop-shape.md
```

Chapter 23: The first sync that checked every script (09-27)

With `assumevalid=0`, bmc synced to the tip in 8 h 0 m at nice 10, evaluating every script, and the harness said FAIL at height 968,807. Recomputing the digest from the two accumulators by hand gave one SHA-256 whose forward hex was bmc's answer and whose reversed hex was Core's. A presentation change on 09-25 had flipped the byte order of the no-height `gettxoutsetinfo` digest. The set had never been wrong (memory note `gettxoutsetinfo-running-digest-is-wrong.md`). As with run 22's torn read in the previous report, a "UTXO set differs from Core" was the checker's fault, and suspecting the checker first found it in an afternoon.

PART VII: THE SCAR TISSUE – patterns, and the rules they left

Chapter 24: Per-something costs that were not per-something

Two of this period's defects are one shape. The reader lane's cost was "per thread", which is fixed only if the threads are. The memory sampler's 26 GB was "per process", which double-counts only if the processes share pages, and they did, 13 times over. In both cases the unit of accounting was right for the design and wrong for one caller. The rule left behind is concrete: any thread-local resource needs a thread-exit release, or its callers must be a pool by construction. And count shared pages once.

Chapter 25: The box as an adversary

- A one-second mount by another project ended a nine-hour Core run (Chapter 5).
- A "stopped" smoke run's children kept syncing for 66 minutes beside Core rerun #6, which then had to be restarted, losing about 1.5 hours. "Stopped" now means no process has its working directory under the bench volume, and that is checked before every timed launch (memory note `stopped-means-no-cwd-under-it.md`).
- The mempool.space backend died for eleven hours on 10-06 because its unit used upstream's development heap of 2,048 MB against a full mempool. It runs with 16 GB now.
- Swap was full for the whole release line, filled by a language-model server sharing the box. Every number in Part V was measured under that load, on both sides.

Chapter 26: The port that said "applies to x86"

The `bmc_osx` tree found real shared defects: the double close (Chapter 8), the CompactSize count (Chapter 20), three testnet4 fixes (#336), mutated compact blocks (#325). Its reports also said four times that a fix applied to x86 when it did not, and one race diagnosis named a thread that does not exist. Every port from it is now revert-checked on x86 and built with gcc `-Werror` before it lands (memory note `porting-bmc-osx-fixes-to-x86.md`).

Chapter 27: The rules now in force

Added in this period, each traceable to a chapter above:

1. A deploy is verified by four things: a new block applied with the oracle's hash; `NRestarts=0` ten minutes in; the RPC clients reconnected; and the descriptor count flat across their traffic (Chapters 8 to 10).
2. Anything that calls into the chain code directly takes the lock the RPC dispatcher would have taken (Chapter 9).
3. Thread-local resources are released at thread exit (Chapters 10, 24).
4. Benchmark datadirs live under the `/srv/nvme8tb` bind mount, and the journal is read for mounts before a drive is blamed (Chapter 5).
5. Before any timed launch, no process has its working directory under the bench volume (Chapter 25).
6. Every benchmark run has a process sampler beside it on both sides (PSS, heap and CPU), with memory lines at IBD end and at ready. Core #6's peak was lost for want of one.
7. Claim apply speed only with the stage row beside it (Chapter 11).
8. Before calling a run download-bound, check the eviction acks and the block gaps (Chapter 12).
9. Bench rows copy Core's loop shape, or say why not (Chapter 22).
10. Ports from the Mac tree are revert-checked on x86 (Chapter 26).

Older rules held and were used constantly: a test is untrusted until it has been watched to fail with its fix reverted; never poll a benchmark node; land batches as `--no-ff` merges; never `git add -A`.

PART VIII: WHAT IS STILL OPEN

- Memory (Chapter 18): the one category Core keeps, by about 2× in PSS.
- Two changes built and merged that day but not yet measured: B12, which switches the header download away from a leader that slows, and B13 (#405, open), which bans a peer only for a second window stall. Run 41 banned 34 of 132 peers, 26 of them newly drawn peers that had not finished their first chunk within Core's 2 s timeout. The next benchmark measures both. The targets: block 1 within 50 s of boot on every sync, and five bans or fewer per run.
- B9 part 1: the ranking's top changes from run to run. 69 to 81 distinct peers served each sync of the release line, against 25 in run 38.
- A4: one 149.5 s `getmininginfo` freeze after a restart into catch-up, on 10-03. Per-step timings have been logged since, and it has not recurred.
- The production unit's descriptor limit is still the default 1,024. Raising it only buys time against a leak. Alerting on the degraded-listener line is the better half and is not done.
- The Esplora facade still starts a thread per connection.

The previous report ended with a node that was probably as fast as Core and measuring equipment that could finally say so. This one ends with a node that is 2.6× faster at the job a node is for, where the measurements of where that time goes are trusted enough that a two-second sleep showed up in them. The next question is the one Part VII keeps asking: whether the next fix gets watched long enough, under real traffic, before it is called done.

APPENDIX A: THE NUMBERS

Benchmark syncs in this period, all on the same machine (AMD Ryzen 9 9950X3D, 123 GiB RAM, one 8 TB NVMe for the datadirs), from genesis, on the public network:

run	date	commit	what changed	bmc	Core	bmc / Core
29	09-21/22	same as 28	a repeat	18:28:54	19:32:54	0.95
30	09-28	1370d041	the repaired link; asm module ports	4:54:30	#4 11:01:00	0.45
31	09-30	49e26354	settings matched to Core	5:28:11	#4	0.50
32	10-01/02	f332e791	16-block chunks (busy box)	6:01:38	—	—
33	10-03/04	f1b7d66c	the six audit fixes	6:01:27	#5 10:30:59	0.57
34	10-05	56bbe8c2	Core's download rules, logged	7:17:39	#6 10:42:05	0.68
37	10-06	8e81ffb5	put fix, RPC lanes, header probe	4:50:52	#6	0.45
38	10-06	020b13dc	the index worker (B4)	4:17:09	#6	0.40
39	10-06	7eb763ab	the JSON arena	4:34:48	#6	0.43
40	10-06/07	b194dd01	B3, B8, M1	4:42:41	#7 9:50:04	0.48
41	10-07	7027c734	B9 part 2, B10, B11, B3 reap	3:48:22	#7 9:50:04	0.39

Runs 30 to 33 are timed to the end of IBD or to the height the Core comparison used (968,987); each report says which. Runs 34 to 41 are timed to the [ready] line, every index at the tip. Run 35 and run 36 were stopped partway, and their wedges are Chapter 12. Every run that finished passed its UTX0 capstone against Core. Sources: docs/reports/ for each run, and docs/reports/2026-10-06-core-vs-bmc-performance-release.md for the release line.

RPC at 32 concurrent clients, median, after the 10-05 and 10-06 work (the release report, §3):

call	bmc	Core
getrawmempool	114 ms	484 ms
getrawtransaction	5 ms	5 ms
getblock v1	9 ms	19 ms
getblock v2, one client	58 ms	87 ms